LATIHAN TERSTRUKTUR

Tugas Pertemuan 4: Implementasi Unit Testing dengan GitHub Actions



Nama : Minarianty Ismail

NIM : 3224010

Prodi : Sistem Informasi

Matkul : Implementasi Pengujian Perangkat Lunak

Tanggal : 09 Oktober 2026

BAGIAN A: GIT & GITHUB DASAR

SOAL 1

1. Perbedaan Centralized VCS dan Distributed VCS

Centralized Version Control System (CVCS) adalah sistem version control yang menggunakan satu server pusat sebagai tempat penyimpanan seluruh versi repository. Developer bekerja dengan mengambil dan mengirim perubahan melalui server pusat. Contoh CVCS adalah SVN dan CVS. Salah satu kelemahannya adalah adanya single point of failure karena ketergantungan terhadap server pusat serta keterbatasan ketika bekerja secara offline.

Distributed Version Control System (DVCS) adalah sistem version control yang memberikan setiap developer salinan lengkap repository. Dengan demikian, developer dapat melakukan pekerjaan secara offline dan proses branching menjadi lebih mudah. Contoh DVCS adalah Git dan Mercurial. Git merupakan salah satu DVCS yang paling banyak digunakan.

Perbedaan utama keduanya terletak pada cara penyimpanan repository. CVCS bergantung pada satu repository pusat, sedangkan DVCS memberikan salinan repository kepada setiap developer.

2. Lima Git Commands yang Wajib Dikuasai Beserta Fungsinya

Lima perintah Git Commands yang penting untuk dikuasai adalah sebagai berikut. Perintah-perintah ini termasuk dalam perintah dasar Git yang dibahas pada materi.

No.	Git Command	Fungsi
1	git init	Menginisialisasi repository Git baru pada folder lokal.
2	git add	Menambahkan file atau perubahan ke dalam staging.
3	git commit	Menyimpan perubahan ke dalam riwayat repository dengan pesan commit tertentu.
4	git push	Mengirim commit dari repository lokal ke repository remote, seperti GitHub.
5	git pull	Me.ngambil perubahan dari reporitory remote ke repository lokal.

3. Fungsi .gitignore dalam Repository Git

.gitignore berfungsi untuk menentukan file atau folder yang tidak perlu dimasukkan atau di commit ke dalam repository Git. File ini digunakan untuk mencegah file yang tidak diperlukan, seperti file hasil build, konfigurasi IDE, cache, atau file tertentu dari sistem operasi, ikut tersimpan di repository. Dalam materi juga diberikan contoh seperti target/ , .idea/ , .vscode/ , *.exec , __pycache__/ , dan .venv/.

SOAL 2

1. Praktik Git Commands

No	Tahap Pelaksanaan	Command yang Digunakan
1	Membuat repository baru pada GitHub dengan nama latihan-git-3224010	Dilakukan melalui GitHub
2	Melakukan clone repository dari GitHub ke komputer lokal	git clone https://github.com/minarianty/latihan-git-3224010.git
3	Berpindah ke direktori repository lokal	cd latihan-git-3224010
4	Memeriksa status repository Git	git status
5	Membuat file README.md dan menuliskan nama	echo Nama: Minarianty Ismail > README.md
6	Menambahkan NIM ke dalam file README.md	echo NIM: 3224010 >> README.md
7	Memeriksa isi file README.md	type README.md
8	Menambahkan file README.md ke staging area	git add readme.md
9	Melakukan commit terhadap file README.md	git commit -m “docs: add README”
10	Mengirim commit ke branch main pada repository GitHub	git push origin main
11	Membuat brach baru dengan nama feature-test	git branch feature-test
12	Memeriksa daftar brach yang tersedia	git branch
13	Berpindah ke brach feature-test	git checkout feature-test
14	Memeriksa brach yang sedang aktif	git branch

15	Membuat file test.txt	echo File pengujian > test.txt
16	Menambahkan file test.txt ke staging area	git add test.txt
17	Melakukan commit terhadap file test.txt	git commit -m “test: add test file”
18	Mengirim branch feature-test ke repository GitHub	git push origin feature-test
19	Berpindah kembali ke branch main	git checkout main
20	Memperbarui repository lokal berdasarkan brach main pada repository GitHub	git pull origin main
21	Menggabungkan brach feature-test ke brach main	git merge feature-test
22	Mengirim hasil merge brach main ke repository GitHub	git push origin main

2. Konfigurasi Identitas Git

Karena pada saat melakuan commit pertama Git belum memiliki identitas pengguna, konfigurasi berikut dilakukan terlebih dahulu:

No	Konfigurasi	Command
1	Menetapkan nama pengguna Git	git config --global user.name “minarianty”
2	Menetapkan Alamat email pengguna Git	git config --global user.email “minarianty20@gmail.com”
3	Memeriksa nama pengguna Git	git config --global user.name
4	Memeriksa alamat email pengguna Git	git config --global user.email

BAGIAN B – UNIT TESTING FRAMEWORK

SOAL 3

Kode program yang diuji:

```
public String validasiPendaftaran(
        String nama,
        int umur,
        String email,
        boolean setujuSyarat
) {
    if (nama == null || nama.isEmpty()) { // P1
        return "Nama harus diisi";
    }
    if (umur < 17) { // P2
        return "Umur minimal 17 tahun";
    }
    if (email == null || !email.contains("@")) { // P3, P4
        return "Email tidak valid";
    }
    if (!setujuSyarat) { // P5
        return "Harus menyetujui syarat dan ketentuan";
    }
    return "Pendaftaran berhasil";
}
```

Kode tersebut melakukan validasi secara berurutan terhadap nama, umur, email, dan persetujuan syarat dan ketentuan. Pola pengujian pada Pertemuan 4 mengharuskan test case dari White-Box Testing diimplementasikan menjadi unit test.

1. **Test Case Berdasarkan Independent Path**

   Pada Tugas Minggu 3, telah disusun enam independent path. Namun, Soal 3 pada Pertemuan 4 meminta lima test case. Oleh karena itu, pengujian pada tugas ini menggunakan Path 1 sampai Path 5 sesuai dengan jumlah yang diminta.

No	Independent Path	Input nama	Input umur	Input email	SetujuSyarat	Expected Output
1	Path 1 – Nama tidak diisi	“”	20	“user@gmail.com”	true	“Nama harus diisi”
2	Path 2 – Umur kurang dari 17 tahun	“Mina”	16	“mina@gmail.com”	true	“Umur minimal 17 tahun”
3	Path 3 – Email bernilai null	“Mina”	20	null	true	“Email tidak valid”
4	Path 4 – Email tidak mengandung @	“Mina”	20	“minagmail.com”	true	“Email tidak valid”
5	Path 5 – Tidak menyetujui syarat	“Mina”	20	“mina@gmail.com”	false	“Harus menyetujui syarat dan ketentuan”

Input tersebut disesuaikan dengan test case yang telah disusun pada Tugas Minggu 3.

2. Implementasi Test Case ke dalam Kode Unit Test JUnit

Karena pada soal hanya diberikan method dan tidak dicantumkan nama class yang membuat method tersebut, contoh berikut menggunakan nama class Pendaftaran sebagai nama class program yang diuji. Nama tersebut perlu disesuaikan dengan nama class yang digunakan pada implementasi program sebenarnya. Kode berikut menggunakan pola Arrange – Act – Assert (AAA) sebagaimana dijelaskan dalam materi pertemuan 4.

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.assertEquals;
public class PendaftaranTest {
    private Pendaftaran pendaftaran;
    @BeforeEach
    public void setUp() {
        pendaftaran = new Pendaftaran();
```

```
}
@Test
public void testPath1_NamaTidakDiisi() {
    // Arrange
    String nama = "";
    int umur = 20;
    String email = "user@gmail.com";
    boolean setujuSyarat = true;
    // Act
    String result = pendaftaran.validasiPendaftaran(
            nama, umur, email, setujuSyarat
    );
    // Assert
    assertEquals("Nama harus diisi", result);
}
@Test
public void testPath2_UmurKurangDari17() {
    // Arrange
    String nama = "Mina";
    int umur = 16;
    String email = "mina@gmail.com";
    boolean setujuSyarat = true;
    // Act
    String result = pendaftaran.validasiPendaftaran(
            nama, umur, email, setujuSyarat
    );
    // Assert
    assertEquals("Umur minimal 17 tahun", result);
}
@Test
public void testPath3_EmailNull() {
    // Arrange
    String nama = "Mina";
```

```
    int umur = 20;
    String email = null;
    boolean setujuSyarat = true;
    // Act
    String result = pendaftaran.validasiPendaftaran(
            nama, umur, email, setujuSyarat
    );
    // Assert
    assertEquals("Email tidak valid", result);
}
@Test
public void testPath4_EmailTidakMengandungAt() {
    // Arrange
    String nama = "Mina";
    int umur = 20;
    String email = "minagmail.com";
    boolean setujuSyarat = true;
    // Act
    String result = pendaftaran.validasiPendaftaran(
            nama, umur, email, setujuSyarat
    );
    // Assert
    assertEquals("Email tidak valid", result);
}
@Test
public void testPath5_TidakMenyetujuiSyarat() {
    // Arrange
    String nama = "Mina";
    int umur = 20;
    String email = "mina@gmail.com";
    boolean setujuSyarat = false;
    // Act
    String result = pendaftaran.validasiPendaftaran(
```

```
            nama, umur, email, setujuSyarat
        );
        // Assert
        assertEquals(
            "Harus menyetujui syarat dan ketentuan",
            result
        );
    }
}
```

3. Hasil Akhir

Komponen	Hasil
Independent Path yang digunakan	5 Path
Test Case	5 Test Case
Unit Test JUnit	5 Method @test
Pola Pengujian	Arrange – Act – Assert (AAA)

Dengan demikian, setiap independent path yang diminta pada Soal 3 Pertemuan 4 telah dikonversi menjadi satu test case dan satu method @Test JUnit. Pendekatan ini sesuai dengan materi yang menjelaskan bahwa test case hasil White-Box Testing dari pertemuan sebelumnya diimplementasikan langsung sebagai unit test pada JUnit.

SOAL 4

Berdasarkan instruksi tugas, Soal 4 meminta tiga hal, yaitu menghitung Cyclomatic Complexity method calculateGrade(), menentukan Independent Path, dan mengimplementasikan unit test JUnit untuk calculateGrade() serta isPassed().

1. Perhitungan Cyclomatic Complexity calculateGrade()

Method:

```
public String calculateGrade(int score) {
    if (score < 0 || score > 100) {
        throw new IllegalArgumentException("Score must be between 0 and 100");
    }
    if (score >= 80) {
```

```
        return "A";
    } else if (score >= 70) {
        return "B";
    } else if (score >= 60) {
        return "C";
    } else if (score >= 50) {
        return "D";
    } else {
        return "E";
    }
}
```

Terdapat 5 Predicate/decision, yaitu:

a. score < 0 | | score > 100

b. score >= 80

c. score >= 70

d. score >= 60

e. score >= 50

Sesuai rumus Cyclomatic Complexity pada materi:

$$CC + Jumlah\ Predicate + 1$$

Maka:

$$CC = 5 + 1$$
$$CC = 6$$

Jadi, Cyclomatic Complexity method calculateGrade() adalah 6, sehingga terdapat 6 independent path yang dapat digunakan sebagai dasar pengujian.

2. Independent Path calculateGrade()

Berdasarkan struktur percabangan program, terdapat independent path.

Path	Kondisi	Expected Output
Path 1	Score < 0 atau score > 100	IllegalArgumentException
Path 2	0 <= score <= 100 dan score >= 80	“A”
Path 3	Score < 80 dan score >= 70	“B”
Path 4	Score < 70 dan score >= 60	“C”
Path 5	Score < 60 dan score >= 50	“D”

Path 6	Score < 50 dan score >= 0	“E”

Bila dituliskan dalam bentuk alur :

a. **Path 1**

   Start → Validasi nilai tidak valid → Exception → End

b. **Path 2**

   Start → Nilai valid → score >= 80 (Ya) → “A” → End

c. **Path 3**

   Start → Nilai valid → score >= 80 (Tidak) → score >= 70 (Ya) → “B” → End

d. **Path 4**

   Start → Nilai valid → score >= 80 (Tidak) → score >= 70 (Tidak) → score >= 60 (Ya) → “C” → End

e. **Path 5**

   Start → Nilai valid → score >= 80 (Tidak) → score >= 70 (Tidak) → score >= 60 (Tidak) → score >= 50 (Ya) → “D” → End

f. **Path 6**

   Start → Nilai valid → score >= 80 (Tidak) → score >= 70 (Tidak) → score >= 60 (Tidak) → score >= 50 (Tidak) → “E” → End

3. Implementasi Unit Test JUnit

Karena kode program pada soal sudah diberikan dengan nama class StudentGrade, maka kode pengujian dibuat dalam class StudentGradeTest.

```
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
public class StudentGradeTest {
    private StudentGrade studentGrade;
    @BeforeEach
    public void setUp() {
        studentGrade = new StudentGrade();
    }
    @Test
    public void testCalculateGrade_InvalidScore() {
        // Arrange
```

```
    int score = 101;
    // Act & Assert
    assertThrows(
        IllegalArgumentException.class,
        () -> studentGrade.calculateGrade(score)
    );
}
@Test
public void testCalculateGrade_GradeA() {
    // Arrange
    int score = 80;
    // Act
    String result = studentGrade.calculateGrade(score);
    // Assert
    assertEquals("A", result);
}
@Test
public void testCalculateGrade_GradeB() {
    // Arrange
    int score = 70;
    // Act
    String result = studentGrade.calculateGrade(score);
    // Assert
    assertEquals("B", result);
}
@Test
public void testCalculateGrade_GradeC() {
    // Arrange
    int score = 60;
    // Act
    String result = studentGrade.calculateGrade(score);
    // Assert
    assertEquals("C", result);
```

```
    }
    @Test
    public void testCalculateGrade_GradeD() {
        // Arrange
        int score = 50;
        // Act
        String result = studentGrade.calculateGrade(score);
        // Assert
        assertEquals("D", result);
    }
    @Test
    public void testCalculateGrade_GradeE() {
        // Arrange
        int score = 49;
        // Act
        String result = studentGrade.calculateGrade(score);
        // Assert
        assertEquals("E", result);
    }
    @Test
    public void testIsPassed_Passed() {
        // Arrange
        int score = 60;
        // Act
        boolean result = studentGrade.isPassed(score);
        // Assert
        assertTrue(result);
    }
    @Test
    public void testIsPassed_NotPassed() {
        // Arrange
        int score = 59;
        // Act
```

```
        boolean result = studentGrade.isPassed(score);
        // Assert
        assertFalse(result);
    }
    @Test
    public void testIsPassed_InvalidScore() {
        // Arrange
        int score = 101;
        // Act & Assert
        assertThrows(
                IllegalArgumentException.class,
                () -> studentGrade.isPassed(score)
        );
    }
}
```

Kode pengujian tersebut menggunakan JUnit 5, sesuai dengan framework unit testing java pada materi, serta menggunakan pola Arrange-Act-Assert (AAA) untuk pengujian yang menghasilkan nilai. Untuk pengujian kondisi exception digunakan assertThrows(), yang juga termasuk assertion JUnit yang dibahas pada materi.

4. Hasil Akhir Soal 4

Komponen	Hasil
Jumlah Predicate calculateGrade()	5
Cyclomatic Complexity	6
Independent Path	6 Path
Test calculateGrade()	6 test case
Test isPassed()	3 test case
Total Unit Test	9 test case

Jadi, jawaban inti soal 4 adalah CC = 6 dan terdapat 6 independent path pada calculateGrade(). Kedua method juga sudah diimplementasikan ke dalam unit test JUnit.

BAGIAN C – GITHUB ACTIONS

SOAL 5

Berdasarkan konfigurasi workflow Github Actions yang diberikan pada soal, analisis dilakukan terhadap nama workflow, trigger atau event, fungsi strategy matrix, serta tahapan yang terdapat dalam job test.

1. **Nama workflow**

   Nama workflow pada konfigurasi YAML tersebut adalah Python CI, sebagaimana ditunjukan pada bagian berikut:

   name: Python CI

   Nama tersebut digunakan sebagai identitas workflow yang ditampilkan pada tab Actions di GitHub.

2. **Trigger / event yang Memicu Workflow**

   Workflow tersebut memiliki dua jenis trigger, yaitu:

   a. **Push**

      yang memicu workflow ketika terdapat perubahan yang dikirim ke branch main atau develop.

```
push:
    branches: [ main, develop ]
```

   b. **Pull Request**

      Yang memicu workflow ketika terdapat pull request yang ditujukan ke branch main.

```
pull_request:
    branches: [ main ]
```

   kedua trigger tersebut memungkinkan pengujian dilakukan secara otomatis ketika perubahan kode dikirim ke branch yang ditentukan atau ketika perubahan diajukan melalui pull request

3. **Fungsi Strategy Matrix**

   Strategy matrix berfungsi untuk menjalankan pengujian secara otomatis pada beberapa konfigurasi yang berbeda. Pada workflow tersebut, pengujian dilakukan menggunakan tiga versi python, yaitu 3.10, 3.11, dan 3.12.

```
strategy:
  matrix:
    python-version: ['3.10', '3.11', '3.12']
```

Dengan konfigurasi tersebut, GitHub Actions menjalankan job pengujian untuk setiap versi Python yang ditentukan. Tujuannya adalah memastikan bahwa program dapat berjalan dengan baik pada berbagai versi Python tersebut serta membantu mendeteksi masalah kompatibilitas. Pengujian untuk ketiga versi ini dapat dijalankan secara paralel sesuai mekanisme matrix build GitHub Actions.

4. Steps yang terdapat dalam Job test

Berdasarkan konfigurasi YAML, terdapat empat blok step yang didefinisikan secara langsung. Jika perintah di dalam bagian instalasi dependensi dipisahkan berdasarkan aktivitasnya, terdapat lima aktivitas utama sebagai berikut.

No	Aktivitas	Fungsi
1	Checkout code	Mengambil kode sumber repository agar tersedia pada runner GitHub Actions menggunakan actions/checkout@v4
2	Setup Python	Menyiapkan versi python sesuai matrix menggunakan actions/setup-python@v5
3	Upgrade pip	Memperbarui package installer Python melalui perintah python – m pip install –upgrade pip.
4	Install depedencies	Menginstal framework pengujian PyTest dan plugin coverage menggunakan pip install pytest pytest-cov.
5	Run test	Menjalankan pengujian sekaligus menghasilkan laporan coverage dalam format XML menggunakan pytest –cov=src –cov—report=xml.

Kelima aktivitas tersebut menggambarkan urutan proses mulai dari menyiapkan kode dan lingkungan Python, memasang dependensi, hingga menjalankan pengujian serta menghasilkan laporan coverage.

SOAL 6

Berdasarkan instruksi tugas, workflow dibuat untuk projek Java dengan nama Java Unit Test, menggunakan runner ubuntu-latest, serta menjalankan pengujian dan menghasilkan laporan code coverage menggunakan Maven dan JaCoCo. Workflow juga harus mengunggah laporan tersebut sebagai artifact bernama jacoco-report. Berikut adalah konfigurasi lengkap file YAML GitHub Actions sesuai ketentuan soal.

Nama file: .github/workflows/test.yml

```
name: Java Unit Test
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Setup JDK 17
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'
      - name: Run tests
        run: mvn test
      - name: Generate coverage report
        run: mvn jacoco:report
      - name: Upload JaCoCo report
        uses: actions/upload-artifact@v4
        with:
          name: jacoco-report
          path: target/site/jacoco/
```

Penjelasan Konfigurasi

No	Komponen	Fungsi
1	name: java unit test	Menentukan nama workflow yang ditampilkan pada tab Actions di GitHub.
2	push	Menjalankan workflow ketika perubahan dikirim ke branch main atau develop.
3	pull_request	Menjalankan workflow ketika terdapat pull request yang ditujukan ke branch main.
4	runs-on: ubuntu-latest	Menentukan lingkungan runner yang digunakan untuk menjalankan workflow.
5	actions/checkout@v4	Mengambil kode dari repository agar dapat diakses oleh runner.
6	actions/setup-java@v4	Menyiapkan JDK 17 dengan distribusi Temurin.
7	mvn test	Menjalankan pengujian otomatis yang dikonfigurasi pada proyek Maven.
8	mvn jacoco:report	Menghasilkan laporan code coverage menggunakan JaCoCo.
9	actions/upload-artifact@v4	Mengunggah direktori laporan JaCoCo sebagai artifact bernama jacoco-report.

Konfigurasi ini mengikuti struktur workflow yang dipelajari pada Pertemuan 4, yaitu nama workflow, trigger, jobs, runner, dan steps.

Hasil Akhir

Workflow Java Unit Test akan berjalan secara otomatis ketika terjadi push ke branch main atau develop, maupun ketika ada pull_request ke branch main. Workflow kemudian menyiapkan JDK 17, menjalankan unit test, menghasilkan laporan code coverage, dan mengunggah laporan tersebut dengan nama artifact jacoco-report. Agar workflow dapat berjalan dengan baik, proyek harus menggunakan Maven dan memiliki konfigurasi plugin JaCoCo pada pom.xml. Setelah workflow berhasil dijalankan, laporan dapat diunduh melalui artifact pada tab Actions di repository GitHub.

BAGIAN D – CODE COVERAGE & INTEGRASI

SOAL 7

1. **Menghitung Total Statement yang Ada di Project**

   Total statement diperoleh dengan menjumlahkan seluruh nilai pada kolom Stmts dalam laporan code coverage.

   Diketahui:

   a. calculator.py = 10 statement

   b. login_system.py = 15 statement

   c. payment.py = 20 statement

   d. utils.py = 5 statement

   Perhitungan:

   Total statement = 10 + 15 + 20 + 5

   Total statement = 50

   Jadi, total statement yang terdapat dalam project adalah 50 statement.

2. **Menghitung Total Statement yang Tidak Tereksekusi (Miss)**

   Total statement yang tidak tereksekusi diperoleh dengan menjumlahkan seluruh nilai pada kolom Miss dalam laporan code coverage.

   Diketahui:

   a. calculator.py = 0 statement

   b. login_system.py = 2 statement

   c. payment.py = 8 statement

   d. utils.py = 1 statement

   Perhitungan:

   Total Miss = 0 + 2 + 8 + 1

   Total Miss = 11

   Jadi, total statement yang tidak tereksekusi adalah 11 statement.

3. Menentukan File dengan Coverage Paling Rendah

Berdasarkan laporan code coverage, persentase coverage setiap file adalah sebagai berikut.

No	Nama File	Stmts	Miss	Coverage
1	calculator.py	10	0	100%
2	login_system.py	15	2	87%
3	payment.py	20	8	60%
4	utils.py	5	1	80%
Total		50	11	78%

Berdasarkan tabel tersebut, file yang memiliki coverage paling rendah adalah payment.py, dengan persentase sebesar 60%.

Diketahui:

Total statement= 20

Miss = 8

Perhitungan:

Statement Tereksekusi = Total Statement – Miss

Statement Tereksekusi = 20 – 8

Statement Tereksekusi = 12

Rumus:

Coverage = (Statement Tereksekusi ÷ Total Statement) x 100%

Coverage = (12 ÷ 20) x 100%

Coverage = 0,6 x 100%

Coverage = 60%

Jadi, file payment.py memiliki coverage paling rendah, yaitu sebesar 60%.

4. Langkah Untuk Meningkatkan Coverage pada File payment.py

Untuk meningkatkan coverage pada file payment.py diperlukan penambahan dan penyempurnaan test case agar statement yang belum tereksekusi dapat tercakup dalam pengujian. Berdasarkan laporan, terdapat 8 dari 20 statement yang belum tereksekusi. Langkah-langkah yang dapat dilakukan adalah sebagai berikut.

a. Memeriksa laporan coverage secara terperinci. Pemeriksaan dilakukan untuk mengetahui baris kode yang belum tereksekusi sehingga bagian program yang memerlukan pengujian tambahan dapat diidentifikasi.

b. Menambahkan test case pada kondisi yang belum diuji. Test case perlu dirancang berdasarkan logika program yang tersedia, termasuk kondisi input yang valid maupun tidak valid serta kemungkinan hasil yang berbeda.

c. Menguji setiap cabang percabangan. Apabila terdapat kondisi if-else, pengujian perlu mencakup setiap cabang yang relevan agar perilaku program pada kondisi yang berbeda dapat diperiksa.

d. Memeriksa hasil pengujian menggunakan assertion. Assertion digunakan untuk memastikan hasil aktual sesuai dengan hasil yang diharapkan, sehingga pengujian tidak hanya menjalankan kode, tetapi juga memverifikasi kebenaran hasilnya.

e. Menjalankan kembali unit test dan memeriksa laporan coverage. Setelah test case ditambahkan, pengujian perlu dilakukan kembali untuk mengetahui peningkatan coverage dan memastikan tidak terdapat pengujian yang gagal.

Dengan langkah-langkah tersebut, jumlah statement yang tereksekusi pada payment.py diharapkan meningkat sehingga cakupan pengujiannya menjadi lebih baik.

5. Apakah Coverage 78% Sudah Cukup Baik? Jelaskan Alasannya!

Menurut saya, coverage sebesar 78% sudah menunjukkan bahwa sebagian besar statement dalam project telah tereksekusi selama pengujian. Namun, nilai tersebut masih perlu ditingkatkan karena belum mencapai target line coverage sebesar 80–90% yang dicantumkan dalam materi Pertemuan 4. Selain itu, cakupan pengujian pada setiap file belum merata. File calculator.py memiliki coverage 100%, sedangkan payment.py hanya mencapai 60%. Kondisi tersebut menunjukkan bahwa masih terdapat bagian program yang memerlukan pengujian tambahan, terutama pada file payment.py yang memiliki 8 statement tidak tereksekusi. Coverage yang tinggi juga tidak secara otomatis menjamin kualitas perangkat lunak. Statement yang tereksekusi belum tentu telah diuji dengan input yang bervariasi atau assertion yang tepat. Oleh karena itu, peningkatan coverage perlu disertai dengan test case yang sesuai, pengujian terhadap kondisi batas, serta pemeriksaan hasil yang diharapkan. Kesimpulannya, coverage 78% dapat menjadi capaian awal yang cukup baik, tetapi belum mencapai target yang dicantumkan dalam materi. Pengujian perlu ditingkatkan, terutama pada file payment.py, agar cakupan pengujian lebih merata dan kualitas perangkat lunak dapat dievaluasi dengan lebih baik.

BAGIAN E – STUDI KASUS TERPADU

SOAL 8

1. Kode Program

Pada soal ini, dibuat kelas BankAccount menggunakan bahasa pemrograman Java. Kelas tersebut memiliki tiga metode utama, yaitu deposit(), withdraw(), dan getBalance().

Validasi diterapkan untuk mencegah penambahan atau penarikan saldo dengan jumlah negatif. Selain itu, penarikan saldo tidak diperbolehkan apabila jumlah yang ditarik melebihi saldo yang tersedia. Jika kondisi tersebut terjadi, program akan menghasilkan IllegalArgumentException.

Kode Program BankAccount.java:

```
package com.stmik;
public class BankAccount {
   private double balance;
   public void deposit(double amount) {
      if (amount < 0) {
         throw new IllegalArgumentException(
            "Amount cannot be negative"
         );
      }
      balance += amount;
   }
   public void withdraw(double amount) {
      if (amount < 0) {
         throw new IllegalArgumentException(
            "Amount cannot be negative"
         );
      }
      if (amount > balance) {
         throw new IllegalArgumentException(
            "Insufficient balance"
         );
      }
```

```
        balance -= amount;
    }
    public double getBalance() {
        return balance;
    }
}
```

2. Implementasi Unit Test

Pengujian unit dilakukan menggunakan framework JUnit 5. Pengujian dirancang untuk memeriksa kondisi normal maupun kondisi yang tidak diperbolehkan, sehingga fungsi utama dan validasi pada kelas BankAccount dapat diperiksa.

Sebanyak 8 test case dibuat, meliputi saldo awal, penyetoran saldo positif, penyetoran negatif, penarikan saldo positif, penarikan negatif, penarikan melebihi saldo, penarikan seluruh saldo, serta penyetoran dengan jumlah nol.

Kode pengujian BankAccountTest.java:

```
package com.stmik;
import org.junit.jupiter.api.BeforeEach;
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
public class BankAccountTest {
    private BankAccount account;
    @BeforeEach
    public void setUp() {
        account = new BankAccount();
    }
    @Test
    public void testInitialBalance() {
        // Arrange dan Act
        double result = account.getBalance();
        // Assert
        assertEquals(0.0, result, 0.001);
    }
    @Test
```

```
public void testDepositPositiveAmount() {
    // Arrange
    double amount = 100000.0;
    // Act
    account.deposit(amount);
    // Assert
    assertEquals(100000.0, account.getBalance(), 0.001);
}
@Test
public void testDepositNegativeAmount() {
    // Arrange
    double amount = -50000.0;
    // Act dan Assert
    assertThrows(
        IllegalArgumentException.class,
        () -> account.deposit(amount)
    );
    assertEquals(0.0, account.getBalance(), 0.001);
}
@Test
public void testWithdrawPositiveAmount() {
    // Arrange
    account.deposit(200000.0);
    // Act
    account.withdraw(50000.0);
    // Assert
    assertEquals(150000.0, account.getBalance(), 0.001);
}
@Test
public void testWithdrawNegativeAmount() {
    // Arrange
    account.deposit(100000.0);
    // Act dan Assert
```

```
    assertThrows(
        IllegalArgumentException.class,
        () -> account.withdraw(-20000.0)
    );
    assertEquals(100000.0, account.getBalance(), 0.001);
}
@Test
public void testWithdrawExceedsBalance() {
    // Arrange
    account.deposit(100000.0);
    // Act dan Assert
    assertThrows(
        IllegalArgumentException.class,
        () -> account.withdraw(150000.0)
    );
    assertEquals(100000.0, account.getBalance(), 0.001);
}
@Test
public void testWithdrawExactBalance() {
    // Arrange
    account.deposit(100000.0);
    // Act
    account.withdraw(100000.0);
    // Assert
    assertEquals(0.0, account.getBalance(), 0.001);
}
@Test
public void testDepositZeroAmount() {
    // Arrange
    account.deposit(0.0);
    // Act
    double result = account.getBalance();
    // Assert
```

```
        assertEquals(0.0, result, 0.001);
    }
}
```

3. Konfigurasi GitHub Actions

GitHub Actions digunakan untuk menjalankan pengujian unit secara otomatis setiap kali terdapat perubahan pada branch main atau develop, serta ketika terdapat pull request menuju branch main.

Workflow menggunakan runner ubuntu-latest dan JDK 17. Pengujian dijalankan melalui perintah mvn test, kemudian laporan code coverage dibuat menggunakan mvn jacoco:report. Hasil laporan disimpan sebagai artifact dengan nama jacoco-report.

Kode workflow bankaccount-ci.yml:

```
name: BankAccount Unit Test
on:
  push:
    branches: [main, develop]
  pull_request:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Setup JDK 17
        uses: actions/setup-java@v4
        with:
          distribution: temurin
          java-version: '17'
      - name: Run unit tests
        run: mvn -f Tugas/Tugas04/BankAccountProject/pom.xml test
      - name: Generate coverage report
        run: mvn -f Tugas/Tugas04/BankAccountProject/pom.xml jacoco:report
```

```
- name: Upload JaCoCo report
  uses: actions/upload-artifact@v4
  with:
    name: jacoco-report
    path: Tugas/Tugas04/BankAccountProject/target/site/jacoco/
    if-no-files-found: error
```

4. Hasil Pengujian dan Code Coverage

Pengujian unit telah dijalankan secara lokal menggunakan Maven. Berdasarkan hasil eksekusi mvn test, diperoleh hasil sebagai berikut.

Keterangan	Hasil
Total test case	8
Test yang berhasil	8
Test yang gagal	0
Error	0
Test yang dilewati	0
Status build	BUILD SUCCESS

Setelah pengujian berhasil, perintah mvn jacoco:report dijalankan untuk menghasilkan laporan code coverage.

Berdasarkan laporan JaCoCo yang dihasilkan, nilai coverage adalah sebagai berikut.

Metrik	Hasil
Instruction coverage	100%
Branch coverage	100%
Instruksi yang terlewat	0 dari 48
Branch yang terlewat	0 dari 6
Baris kode	12
Metode	4
Kelas	1

Hasil tersebut menunjukkan bahwa seluruh instruksi dan percabangan yang dianalisis oleh JaCoCo telah tercakup oleh pengujian. Meskipun demikian, nilai coverage 100% tidak secara otomatis menjamin bahwa program bebas dari kesalahan karena coverage menunjukkan cakupan eksekusi kode, bukan keseluruhan kualitas atau kebenaran program.

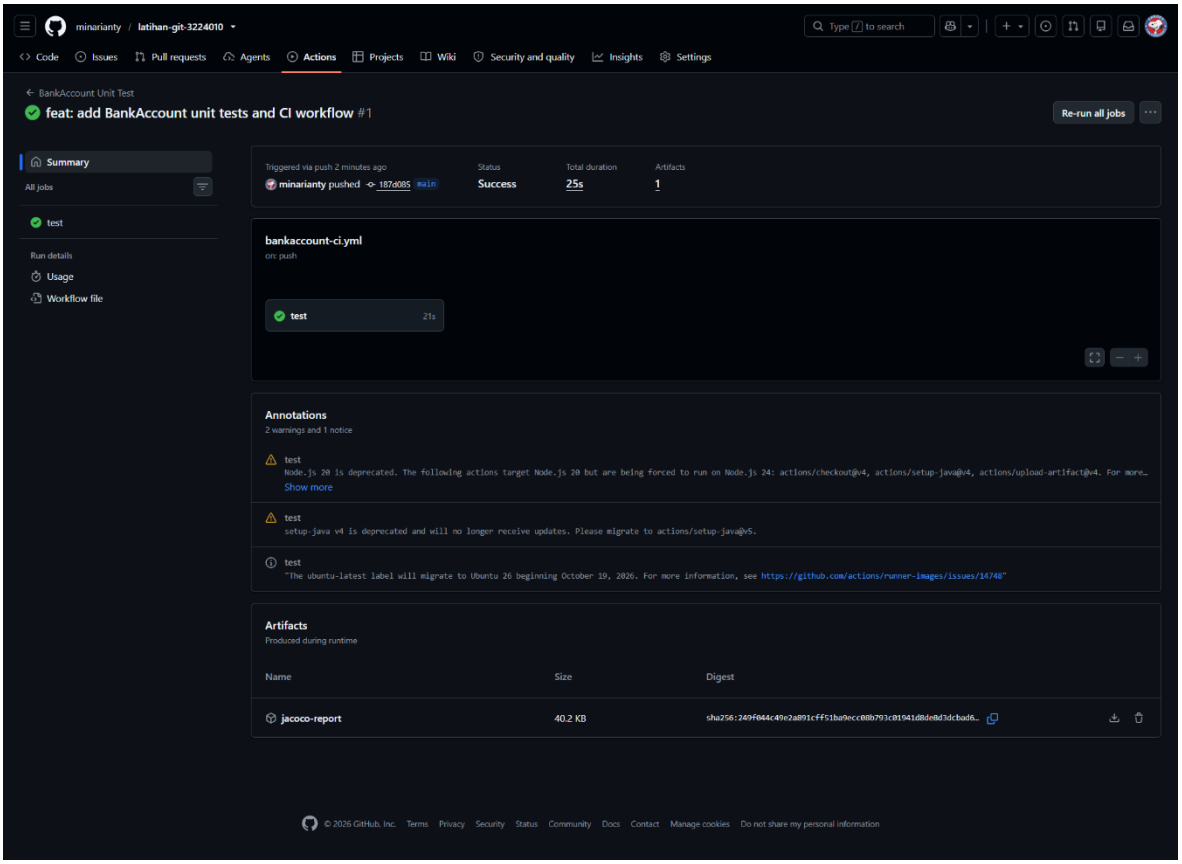
5. Hasil Implementasi GitHub Actions

Proyek telah diunggah ke repository GitHub dan workflow berhasil dijalankan. Pada halaman GitHub Actions, status eksekusi menunjukkan Success, dengan job test yang berhasil diselesaikan. Workflow juga menghasilkan artifact jacoco-report yang dapat diunduh untuk memeriksa laporan coverage.

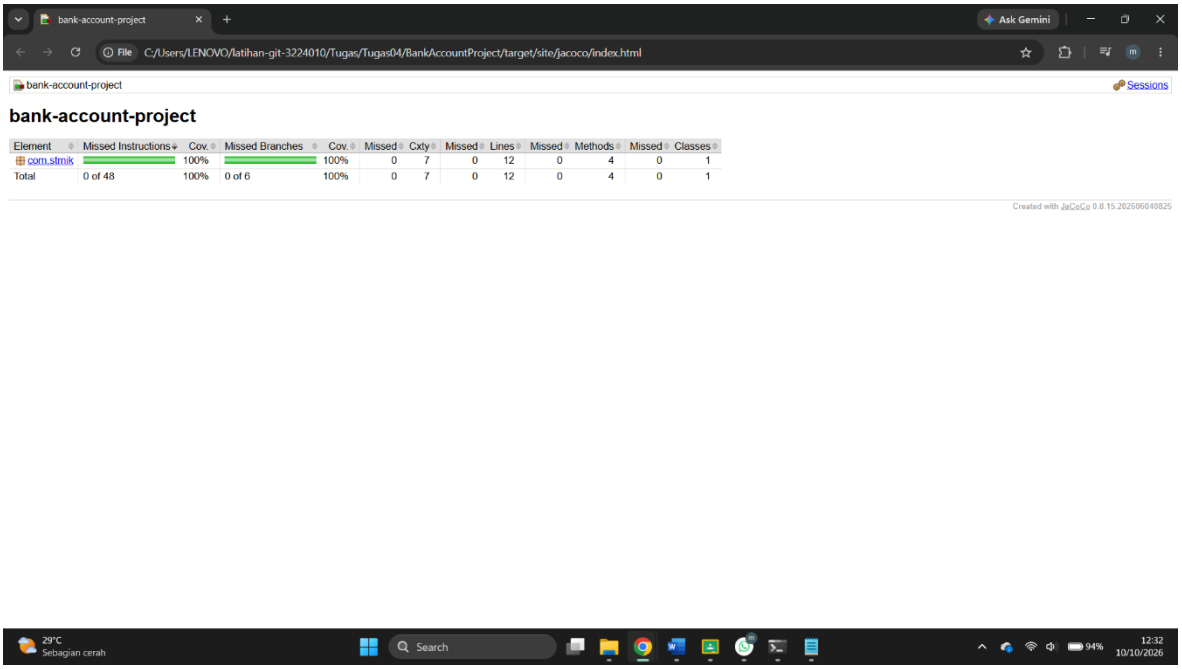
Dengan demikian, implementasi telah mencakup pembuatan kelas BankAccount, pengujian unit menggunakan JUnit 5, otomatisasi pengujian melalui GitHub Actions, serta pembuatan dan penyimpanan laporan code coverage menggunakan JaCoCo.

Repository GitHub:

https://github.com/minarianty/latihan-git-3224010



Screenshot pipeline GitHub Actions yang menunjukkan status Success, job test berhasil, dan artifact jacoco-report.



Screenshot laporan code coverage JaCoCo yang menunjukkan instruction coverage dan branch coverage masing-masing sebesar 100%.